```
!pip -q install scikit-image opencv-contrib-python-headless

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files
from skimage.feature import hog
from skimage import exposure
import os

print("All libraries imported successfully!")
print("OpenCV version:", cv2.__version__)
print("Ready to start Experiment 5.")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 69.5/69.5 MB 10.4 MB/s eta 0:00:00
All libraries imported successfully!
OpenCV version: 5.0.0
Ready to start Experiment 5.
```

```
uploaded = files.upload()

if not uploaded:
    raise ValueError("Please upload an image.")

filename = next(iter(uploaded))
image_bytes = np.frombuffer(uploaded[filename], np.uint8)

img = cv2.imdecode(image_bytes, cv2.IMREAD_COLOR)

if img is None:
    raise ValueError("Invalid image file. Please upload JPG or PNG.")

# Large images ko resize karna
height, width = img.shape[:2]
scale = min(1.0, 1000 / max(height, width))

if scale < 1.0:
    img = cv2.resize(
        img, None, fx=scale, fy=scale,
        interpolation=cv2.INTER_AREA
    )

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Image loaded successfully!")
print("Image name:", filename)
print("Image dimensions:", gray.shape)

plt.figure(figsize=(7, 5))
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()
```

Choose files nature.jpg

nature.jpg(image/jpeg) - 27940 bytes, last modified: 13/10/2026 - 100% done

```
Saving nature.jpg to nature (2).jpg
Image loaded successfully!
Image name: nature (2).jpg
Image dimensions: (460, 612)
```




```
if not hasattr(cv2, "SIFT_create"):
    raise RuntimeError(
        "SIFT is unavailable. Restart the Colab session, "
        "then run Cell 1 and Cell 2 again."
    )

sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

sift_image = cv2.drawKeypoints(
    gray,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

print("SIFT feature extraction completed.")
print("Number of keypoints:", len(keypoints))

if descriptors is None:
    print("No SIFT descriptors found. Try a clearer image.")
else:
    print("SIFT descriptor shape:", descriptors.shape)

plt.figure(figsize=(9, 6))
plt.imshow(sift_image, cmap="gray")
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

```
SIFT feature extraction completed.
Number of keypoints: 306
SIFT descriptor shape: (306, 128)
```




```
hog_features, hog_image = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True,
    feature_vector=True
)

hog_visual = exposure.rescale_intensity(
    hog_image,
    in_range=(0, max(float(np.max(hog_image)), 1e-8))
)

print("HOG feature extraction completed.")
print("HOG feature vector length:", len(hog_features))

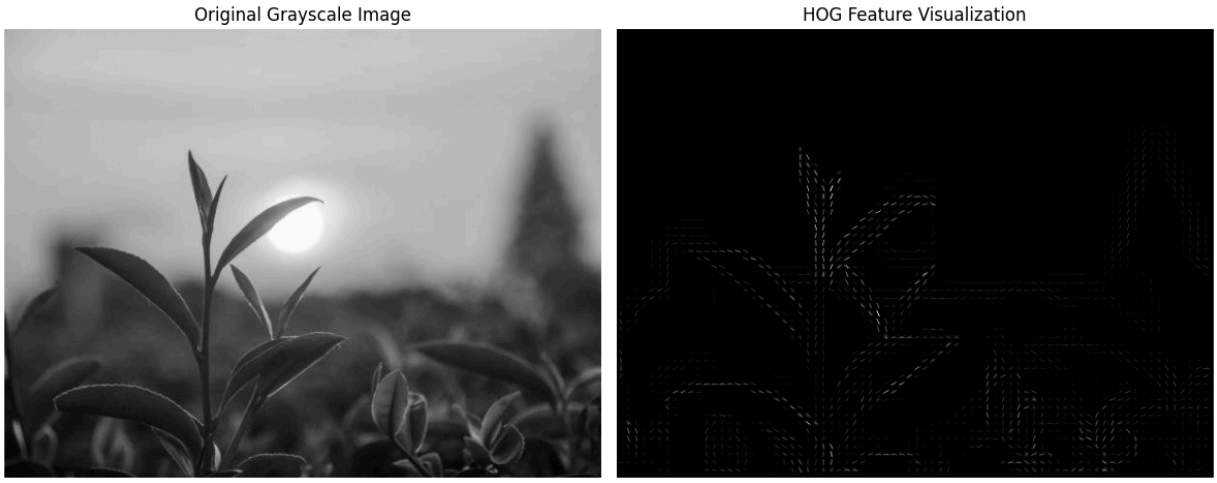
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(hog_visual, cmap="gray")
plt.title("HOG Feature Visualization")
plt.axis("off")

plt.tight_layout()
plt.show()
```

```
HOG feature extraction completed.
HOG feature vector length: 151200
```




```
print("=" * 45)
print("SIFT AND HOG COMPARISON")
print("=" * 45)

print("Image dimensions:", gray.shape)
print("SIFT keypoints:", len(keypoints))

if descriptors is not None:
    print("SIFT descriptor shape:", descriptors.shape)
else:
    print("SIFT descriptors: Not available")

print("HOG feature vector length:", len(hog_features))

print("\nSIFT:")
print("- Detects distinctive local keypoints.")
print("- Useful for matching similar images.")
print("- Handles image rotation and scale changes to a degree.")

print("\nHOG:")
print("- Represents edge and gradient orientation patterns.")
print("- Useful for shape-based object detection.")
print("- Commonly used with a classifier.")
```

```
=============================================
SIFT AND HOG COMPARISON
=============================================
Image dimensions: (460, 612)
SIFT keypoints: 306
SIFT descriptor shape: (306, 128)
HOG feature vector length: 151200

SIFT:
- Detects distinctive local keypoints.
- Useful for matching similar images.
- Handles image rotation and scale changes to a degree.

HOG:
- Represents edge and gradient orientation patterns.
- Useful for shape-based object detection.
- Commonly used with a classifier.
```

```
# SIFT Feature Matching - Fixed Version

from google.colab import files
```

```
from google.colab import files
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Check first image
if 'gray' not in globals():
    raise ValueError("Please run Cell 2 first to upload the first image.")

# Upload only the second image
print("Select ONE second image.")
uploaded_second = files.upload()

if len(uploaded_second) == 0:
    raise ValueError("No image uploaded. Run this cell again.")

second_name = next(iter(uploaded_second))
data = np.frombuffer(uploaded_second[second_name], np.uint8)
img2_color = cv2.imdecode(data, cv2.IMREAD_COLOR)

if img2_color is None:
    raise ValueError("Invalid image. Please upload a JPG or PNG.")

img1 = gray.copy()
img2 = cv2.cvtColor(img2_color, cv2.COLOR_BGR2GRAY)

# Resize second image if necessary
h, w = img2.shape
scale = min(1.0, 1000 / max(h, w))

if scale < 1.0:
    img2 = cv2.resize(
        img2, None, fx=scale, fy=scale,
        interpolation=cv2.INTER_AREA
    )

# Extract SIFT features
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)

print("First image keypoints:", len(kp1))
print("Second image keypoints:", len(kp2))

if des1 is None or des2 is None or len(des1) < 2 or len(des2) == 0:
    print("Not enough features for matching.")
    print("Try clearer images with more visible details.")
else:
    matcher = cv2.BFMatcher(cv2.NORM_L2)
    pairs = matcher.knnMatch(des1, des2, k=2)

    good_matches = []
    for pair in pairs:
        if len(pair) == 2:
            m, n = pair
            if m.distance < 0.75 * n.distance:
                good_matches.append(m)

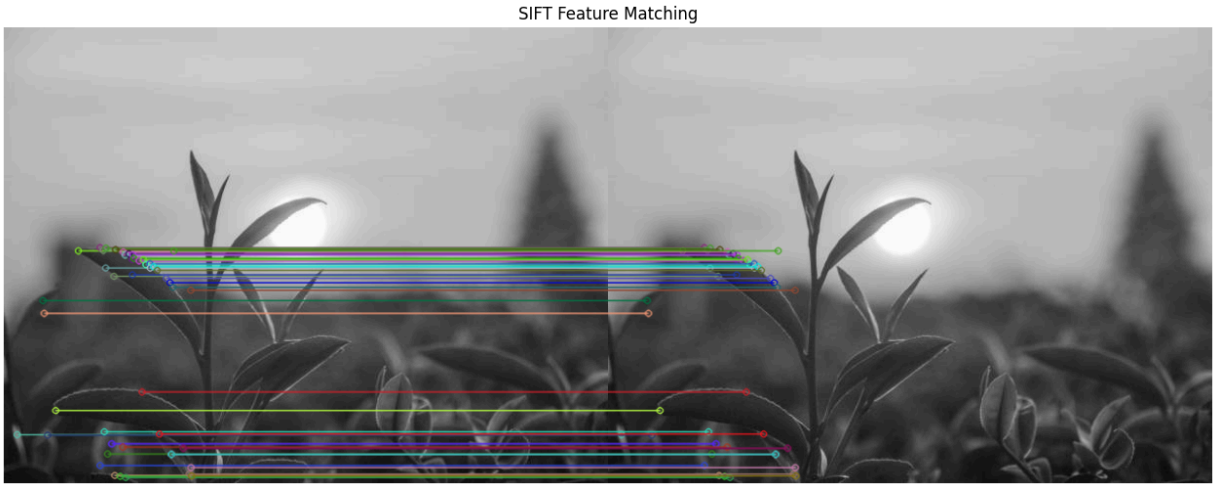
    result = cv2.drawMatches(
        img1, kp1,
        img2, kp2,
        good_matches[:50], None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    print("Good matches found:", len(good_matches))

    plt.figure(figsize=(16, 7))
```

```
    plt.imshow(result, cmap="gray")
    plt.title("SIFT Feature Matching")
    plt.axis("off")
    plt.show()
```

```
Select ONE second image.
Choose files nature.jpg
nature.jpg(image/jpeg) - 27940 bytes, last modified: 13/10/2026 - 100% done
Saving nature.jpg to nature (5).jpg
First image keypoints: 306
Second image keypoints: 306
Good matches found: 306
```




```
cv2.imwrite("sift_keypoints.png", sift_image)
cv2.imwrite("hog_visualization.png", hog_visual)

print("Output images saved successfully.")

files.download("sift_keypoints.png")
files.download("hog_visualization.png")
```

```
Output images saved successfully.
```

```
questions_answers = [
    (
        "Q1. What is feature extraction in image processing?",
        "Feature extraction is the process of identifying useful "
        "information from an image, such as edges, corners, shapes "
        "and textures, for further analysis."
    ),
    (
        "Q2. What is SIFT?",
        "SIFT stands for Scale-Invariant Feature Transform. "
        "It detects distinctive keypoints and creates descriptors "
        "that help identify similar features across images."
    ),
    (
        "Q3. What are keypoints and descriptors?",
        "Keypoints are important locations in an image. "
        "Descriptors are numerical representations of the local "
        "image regions around those keypoints."
    ),
    (
        "Q4. What is HOG?",
        "HOG stands for Histogram of Oriented Gradients. "
        "It represents an image by calculating the distribution "
```

```
        "It represents an image by calculating the distribution "
        "of gradient directions in local image regions."
    ),
    (
        "Q5. Compare SIFT and HOG.",
        "SIFT detects local keypoints and describes their surrounding "
        "regions, making it useful for feature matching. HOG describes "
        "edge orientations across image regions and is useful for "
        "shape-based object detection."
    ),
    (
        "Q6. What is meant by scale and rotation invariance?",
        "Scale invariance means features can be recognized at "
        "different sizes. Rotation invariance means features can "
        "remain recognizable when an image is rotated. SIFT is "
        "designed to provide robustness to both changes."
    ),
    (
        "Q7. What are the applications of HOG?",
        "HOG is used in object detection, pedestrian detection, "
        "shape analysis and computer vision classification tasks."
    ),
    (
        "Q8. Why is feature extraction important?",
        "Feature extraction converts image information into useful "
        "numerical representations. These features help machine "
        "learning algorithms classify images, detect objects "
        "and recognize patterns."
    ),
    (
        "Q9. Compare handcrafted features with deep features.",
        "Handcrafted features such as SIFT and HOG use predefined "
        "algorithms to describe images. Deep features are learned "
        "automatically from data by neural networks such as CNNs."
    ),
    (
        "Q10. How do SIFT and HOG support image matching and recognition?",
        "SIFT supports matching by comparing local feature descriptors "
        "between images. HOG provides information about shapes and "
        "edge patterns that can be used by classifiers for object "
        "detection and recognition."
    )
]

for question, answer in questions_answers:
    print("\n" + question)
    print("Answer:", answer)
    print("-" * 80)
```

```
Q1. What is feature extraction in image processing?
Answer: Feature extraction is the process of identifying useful information from an image, suc
--------------------------------------------------------------------------------

Q2. What is SIFT?
Answer: SIFT stands for Scale-Invariant Feature Transform. It detects distinctive keypoints an
--------------------------------------------------------------------------------

Q3. What are keypoints and descriptors?
Answer: Keypoints are important locations in an image. Descriptors are numerical representatio
--------------------------------------------------------------------------------

Q4. What is HOG?
Answer: HOG stands for Histogram of Oriented Gradients. It represents an image by calculating
--------------------------------------------------------------------------------

Q5. Compare SIFT and HOG.
Answer: SIFT detects local keypoints and describes their surrounding regions, making it useful
```

```
--------------------------------------------------------------------------------

Q6. What is meant by scale and rotation invariance?
Answer: Scale invariance means features can be recognized at different sizes. Rotation invaria
--------------------------------------------------------------------------------

Q7. What are the applications of HOG?
Answer: HOG is used in object detection, pedestrian detection, shape analysis and computer vis
--------------------------------------------------------------------------------

Q8. Why is feature extraction important?
Answer: Feature extraction converts image information into useful numerical representations. T
--------------------------------------------------------------------------------

Q9. Compare handcrafted features with deep features.
Answer: Handcrafted features such as SIFT and HOG use predefined algorithms to describe images
--------------------------------------------------------------------------------

Q10. How do SIFT and HOG support image matching and recognition?
Answer: SIFT supports matching by comparing local feature descriptors between images. HOG prov
--------------------------------------------------------------------------------
```